The Programming Framework: A General Method for Process Analysis Using LLMs and Mermaid Visualization

Gary Welz
Retired Faculty Member
John Jay College, CUNY (Department of Mathematics and Computer Science)
Borough of Manhattan Community College, CUNY
CUNY Graduate Center (New Media Lab)
Email: garywelz@gmail.com

Abstract

Scientific and technical fields rely heavily on textual process descriptions that are difficult to compare, analyze, or reuse computationally. We introduce the Programming Framework, a methodology for transforming textual process descriptions into structured, computable flowcharts using large language models (LLMs) and Mermaid diagram syntax. The framework suggests a five-category color-coding system (Red: triggers/inputs, Yellow: structures/objects, Green: processing/operations, Blue: intermediates/states, Violet: products/outputs) for consistent representation across disciplines, with customization for domain-specific needs. We demonstrate effectiveness through application across biology (100+ processes via the Genome Logic Modeling Project), chemistry (70+ processes), mathematics, physics, and computer science. The methodology enables researchers, educators, and AI systems to visualize, compare, and optimize processes across scientific disciplines. The Programming Framework is designed as reusable infrastructure that others can adopt, extend, and critique, with all methodology, tools, and examples publicly available.

Keywords: process visualization, LLM-powered analysis, Mermaid diagrams, knowledge representation, cross-disciplinary analysis, scientific workflows

---

1. Introduction

1.1 Motivation

Processes are central to scientific and technical work. Biologists describe reaction pathways and regulatory networks; computer scientists specify algorithms and data pipelines; chemists outline reaction mechanisms; physicists model physical processes; mathematicians formalize proofs and algorithms. These processes are usually documented in prose, sometimes accompanied by diagrams that are static, informal, or discipline-specific.

This creates several problems:

1. **Limited comparability:** Two textual descriptions of similar processes may be difficult to compare systematically, even within the same discipline.
2. **Low reusability:** Process knowledge is not readily available as structured data for downstream tools (e.g., simulation, search, or automated checking).
3. **Inconsistent notation:** Disciplines use different diagramming conventions, reducing cross-domain transfer and interdisciplinary collaboration.
4. **High human labor cost:** Manually authoring and maintaining high-quality process diagrams is time-consuming and error-prone.

Large language models (LLMs) now provide a pragmatic way to transform text into structured representations. However, without a clear methodology and standardized representation format, attempts at "LLM → diagram" conversions risk being ad-hoc and irreproducible.

1.2 The Programming Framework Approach

The Programming Framework addresses these challenges by providing:

1. **A suggested color-coding system** as a starting point, with flexibility for domain-specific customization
2. **A repeatable LLM-based methodology** for extracting processes from text into structured flowcharts
3. **Mermaid Markdown as visualization standard** for human-readable, version-controllable diagrams
4. **A JSON-based storage format** with metadata for managing large collections of process diagrams
5. **An iterative refinement loop** where humans and AI collaborate to improve process fidelity over time

The Framework is intended as a "meta-tool": it does not prescribe what processes must look like in each field, but instead offers a consistent way to represent and refine them, enabling cross-disciplinary comparison and optimization.

1.3 Contributions

This paper contributes:

Methodological: - A repeatable LLM-based pipeline for extracting processes from text into structured flowcharts - A concrete, step-by-step methodology with prompt engineering guidelines and validation workflows - Practical observations about prompt design, failure modes, and human–AI collaboration in process modeling

Representational: - A suggested five-category color-coding system (Red/Yellow/Green/Blue/Violet) as a starting point, with examples of domain-specific customization (e.g., GLMP's logical connectives) - Mermaid + JSON as computable, version-controlled artifacts with extensible metadata schemas

Empirical: - Demonstrated application across biology (GLMP with 100+ processes), mathematics, physics, chemistry, and computer science - Working systems and live artifacts demonstrating cross-domain transferability

---

2. Related Work

This section situates the Programming Framework within existing research on process modeling, knowledge representation, and LLM-based diagram generation.

2.1 Process Modeling in Scientific Domains

Process modeling has a long history in scientific computing and knowledge representation. In biology, standards like SBGN [5], BioPAX [6], and SBML [7] provide formal representations for biochemical networks, enabling quantitative modeling and simulation. However, these standards require specialized tools (CellDesigner [8], Cytoscape [9]) and significant expertise, creating barriers to adoption for rapid visualization and cross-domain comparison.

In computer science, UML [10] provides comprehensive notation for software processes, but its complexity can be overwhelming for simple process flows. Chemistry lacks a unified process visualization standard

comparable to SBGN, with tools primarily focused on molecular structure drawing rather than reaction pathways. Mathematics and physics similarly lack standardized process visualization formats, relying on domain-specific simulation tools.

The Programming Framework addresses a gap: providing accessible, text-based process visualization that works across domains while maintaining compatibility with existing standards when needed. This work builds on principles from knowledge representation (Gruber, 1993) and ontology engineering (Noy & McGuinness, 2001), adapting them for practical, accessible process visualization.

2.2 Knowledge Representation and Ontology Engineering

The Framework's JSON-based metadata schema draws from principles in ontology engineering and semantic web technologies. While not implementing full RDF/OWL semantics, the structured metadata approach enables entity linking, cross-referencing, and integration with knowledge graphs—core goals of semantic web research (Berners-Lee et al., 2001). The approach aligns with lightweight ontology principles (Uschold & Gruninger, 1996), prioritizing practical utility over formal completeness.

The color-coding system provides a lightweight ontology for process stages (input $\rightarrow$ structure $\rightarrow$ operation $\rightarrow$ intermediate $\rightarrow$ output), enabling pattern recognition and cross-domain comparison without requiring formal ontology development. This aligns with research on visual knowledge representation (Larkin & Simon, 1987) and the use of color semantics for information organization (Healey & Enns, 2012).

2.3 LLM-Based Knowledge Extraction

Recent advances in large language models have enabled automated extraction of structured information from unstructured text (Brown et al., 2020; OpenAI, 2023). The Programming Framework applies these capabilities specifically to process extraction, building on work in relation extraction (Zelenko et al., 2003), event extraction (Ahn, 2006), and knowledge graph construction (Ji et al., 2021).

Unlike general-purpose knowledge extraction, the Framework focuses on process-specific structures: sequences, decisions, branches, and flows. This specialization enables more reliable extraction through domain-agnostic but process-aware prompts, drawing on research in structured output generation (Schulman et al., 2022) and prompt engineering for scientific applications (White et al., 2023).

2.4 Scientific Workflow Systems

Scientific workflow systems (e.g., Taverna, Galaxy, KNIME) enable researchers to compose and execute computational pipelines. The Programming Framework complements these systems by providing visualization and analysis of process logic, rather than execution environments. The Framework's diagrams could potentially be mapped to workflow specifications, enabling bidirectional translation between visual process descriptions and executable workflows.

2.5 Diagram Generation and Visualization

Automated diagram generation from text has been explored in various contexts (Kong et al., 2019; Liu et al., 2020). The Programming Framework's contribution is its focus on scientific processes, cross-domain applicability, and integration with version-controlled, structured data formats. The use of Mermaid Markdown provides a balance between expressiveness and accessibility, avoiding the complexity of specialized diagramming languages while maintaining sufficient structure for computational analysis.

---

3. Methodology

The Programming Framework treats process extraction as a structured transformation task with human-in-the-loop verification. At a high level, the pipeline has five stages:

1. Process scoping and selection
2. LLM-based structure extraction
3. Mermaid diagram generation with color coding
4. JSON storage and metadata attachment
5. Iterative refinement (human + AI)

3.1 Process Scoping and Selection

The input to the Framework is a text source that describes a process. This can be:

1. A paragraph from a research article (e.g., a "Methods" section)
2. A textbook excerpt describing a pathway or algorithm
3. A protocol or step-by-step procedure
4. A conceptual description of a system or workflow

The scoping step consists of:

1. **Identify a single process:** Select a coherent process rather than mixing multiple distinct processes
2. **Define the level of granularity:** Decide whether the diagram should be high-level (5–10 nodes) or detailed (20+ nodes)
    - **High-level (5–10 nodes):** Suitable for educational overviews, conceptual understanding, initial exploration, or when the source text provides only a summary description
    - **Detailed (20+ nodes):** Appropriate for technical protocols, comprehensive process documentation, research methods sections, or when precise step-by-step accuracy is required
3. **Specify the perspective:** For example, "molecular events," "data flow," "user actions," or "computational steps"

These scoping decisions are made by a human operator but are clearly communicated to the LLM via the prompt.

3.2 Suggested Color-Coding System

The Programming Framework suggests a five-category color-coding system as a starting point for process visualization. This system provides a baseline semantic meaning that can be adapted for specific domains:

- **Red (#ff6b6b):** Triggers & Inputs - Initial conditions, environmental inputs, starting materials, or data inputs (uses white text for contrast)
- **Yellow (#ffd43b):** Structures & Objects - Physical structures, molecules, data structures, algorithms, or logical constructs (uses black text for readability)
- **Green (#51cf66):** Processing & Operations - Transformations, reactions, computations, or operations that change state (uses white text for contrast)
- **Blue (#74c0fc):** Intermediates & States - Intermediate products, temporary states, or transitional conditions (uses white text for contrast)

- **Violet (#b197fc):** Products & Outputs - Final outputs, end products, or results (uses white text for contrast)

Rationale for Five Categories: The five-category system balances cognitive load with visual distinctiveness. Five colors provide sufficient semantic granularity to capture the primary stages of most processes (input → structure → operation → intermediate → output) while remaining visually distinguishable and memorable. This number avoids both oversimplification (fewer categories would lose important distinctions) and cognitive overload (more categories would reduce rapid visual parsing). The color choices (Red, Yellow, Green, Blue, Violet) follow a natural spectrum that aids memorability and intuitive association with process stages.

Accessibility Considerations: The color-coding system may present challenges for colorblind users. While the Framework allows customization and alternative visual indicators (shapes, patterns, labels), the default five-color scheme should be supplemented with text labels and, when possible, shape or pattern variations for accessibility.

Node Shape Alternatives for Accessibility: Mermaid supports different node shapes that can serve as secondary identifiers independent of color: - **Ovals/Stadium shapes** for triggers/inputs: `A([Input])` - **Rectangles** for structures/objects: `B[Structure]` - **Hexagons** for processing/operations: `C{{Operation}}` - **Cylinders** for intermediates/states: `D[(State)]` - **Trapezoids** for products/outputs: `E[/Output\]`

Example Mermaid code using shape-based encoding:

```
flowchart TD
    A([Input]) --> B[Structure]
    B --> C{{Operation}}
    C --> D[(Intermediate)]
    D --> E[/Output\]
```

This shape-based approach provides accessibility for colorblind users while maintaining the visual distinction of process stages. Future work should include systematic evaluation of colorblind accessibility and development of alternative visual encoding schemes.

Customization and Adaptation: The color scheme is not rigidly enforced. Different processes may require different color assignments based on domain-specific needs. For example, in the Genome Logic Modeling Project (GLMP), a custom color scheme is used where specific colors represent logical connectives (AND, OR, NOT nodes) to better represent regulatory logic in biological systems. Other processes might benefit from domain-specific color schemes that highlight particular aspects of the process (e.g., energy levels, time sequences, or hierarchical relationships).

The suggested color system can enable: - **Rapid visual parsing** of process flow (when consistently applied) - **Cross-disciplinary comparison** using consistent semantics (within a shared scheme) - **Pattern recognition** across different domains - **Domain-specific customization** for specialized visualization needs

3.3 LLM Prompt Engineering for Process Extraction

The core of the method uses LLMs to convert scoped text into an ordered set of process elements. A typical extraction prompt has the following structure:

1. **Task statement:**
   "You are a scientific process modeler. Given a textual description of a process, extract a sequence of steps, decisions, and flows. Optionally classify each element according to the Programming

Framework's suggested color system: Red (triggers/inputs), Yellow (structures/objects), Green (processing/operations), Blue (intermediates/states), Violet (products/outputs). Note that domain-specific processes may require custom color assignments."

2. **Output schema:**
   Request a structured JSON-like or list-form output with elements such as:
   - `id`: step identifier
   - `type`: `"trigger" | "structure" | "operation" | "intermediate" | "product"`
   - `label`: concise description
   - `inputs` / `outputs`: optional fields for data or entities
   - `color`: assigned color category
3. **Constraints:**
   - No speculative steps beyond the text
   - Use consistent terminology for entities
   - Explicitly represent branching conditions
   - Maintain color consistency throughout
4. **Example:**
   Provide 1–2 worked examples to anchor the model's behavior, showing how different process types map to color categories.

Given the scoped text and this prompt, the LLM produces a candidate structured representation with color assignments.

Prompt Sensitivity: Prompt design significantly influences extraction fidelity; this framework therefore treats prompts as first-class methodological artifacts. Small variations in prompt wording, example selection, or constraint specification can yield substantially different outputs. The Framework addresses this through iterative refinement and validation workflows, acknowledging that prompt engineering is an ongoing process rather than a one-time configuration.

3.4 Mermaid Syntax and Structure

Mermaid is an open-source JavaScript-based diagramming and charting software that generates diagrams from text-based descriptions, created by Knut Sveidqvist in 2014 [1]. The project originated from a need to simplify diagram creation in documentation workflows and has since become widely adopted, with native support in GitHub, GitLab, Notion, Obsidian, and many other platforms [1,2]. The Framework primarily uses Mermaid flowcharts with color styling. Mermaid's text-based syntax enables version control, collaborative editing, and seamless integration into documentation systems.

Figure 1: Basic Programming Framework Structure. Color Legend: Red (#ff6b6b) = Triggers/Inputs, Yellow (#ffd43b) = Structures/Objects, Green (#51cf66) = Processing/Operations, Blue (#74c0fc) = Intermediates/States, Violet (#b197fc) = Products/Outputs. Note: This is one possible representation; more detailed versions can be generated by specifying greater granularity in the prompt.

Mermaid Markdown Code:

```
flowchart TD
    A[Start/Input] --> B[Process Step]
    B --> C{Decision Point}
```

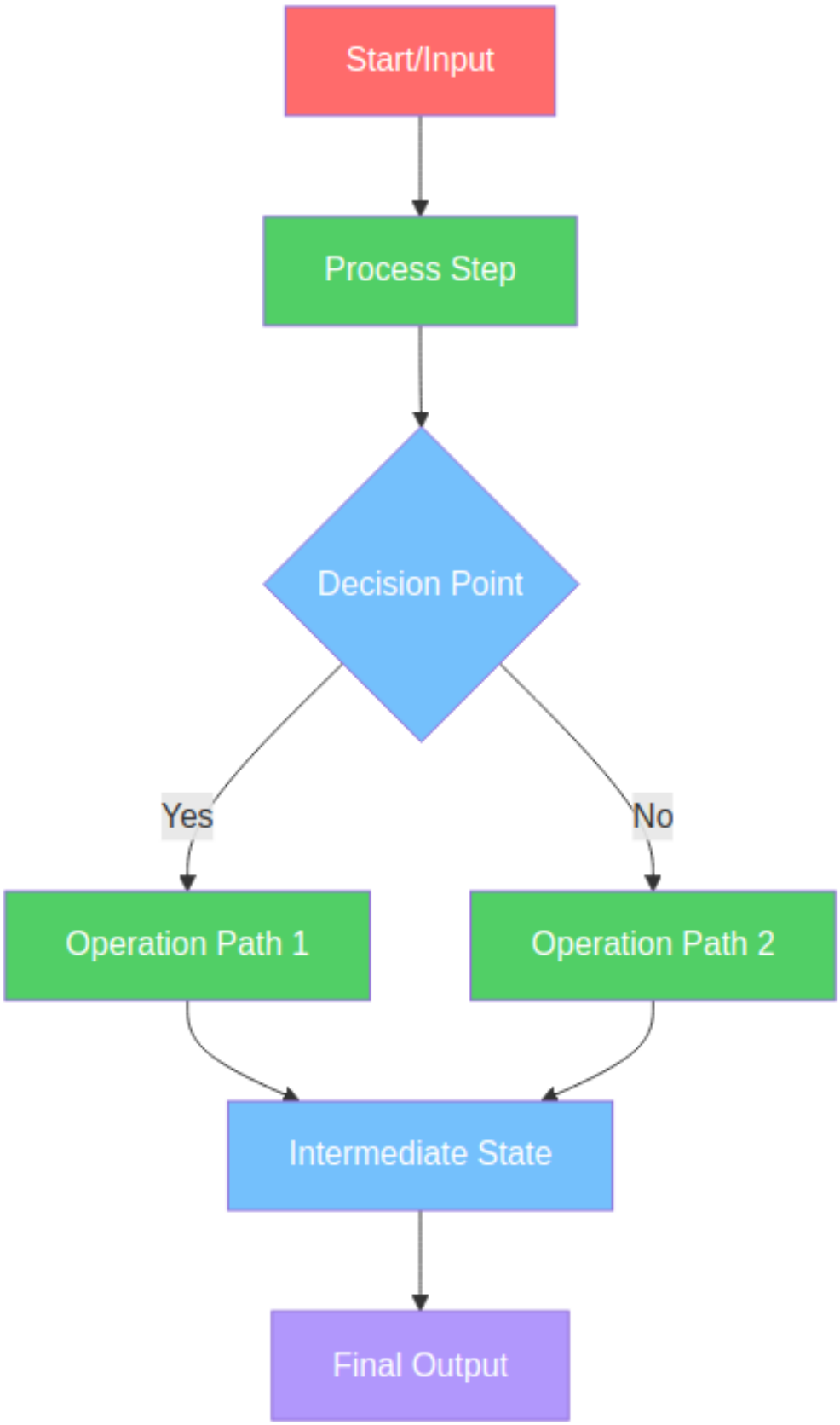

Figure 1: Figure 1: Basic Programming Framework Structure

```
C -->|Yes| D[Operation Path 1]
C -->|No| E[Operation Path 2]
D --> F[Intermediate State]
E --> F
F --> G[Final Output]

style A fill:#ff6b6b,color:#fff
style B fill:#51cf66,color:#fff
style C fill:#74c0fc,color:#fff
style D fill:#51cf66,color:#fff
style E fill:#51cf66,color:#fff
style F fill:#74c0fc,color:#fff
style G fill:#b197fc,color:#fff
```

The transformation from LLM output to Mermaid involves:

1. **Node mapping:** Assign each extracted step or decision to a Mermaid node with a short label
2. **Edge construction:** Translate the LLM's ordering and branching into `-->` or labeled edges (e.g., `-->|Yes|`)
3. **Color application:** Apply the appropriate color styling based on the Framework's five-category system
4. **Layout specification:** Standardize on top-down (`TD`) or left-right (`LR`) layouts for consistency
5. **Styling and grouping (optional):** Use Mermaid's subgraphs or classes to group related steps or highlight categories

This transformation can be done by the same LLM (with a different prompt) or by deterministic code that maps a structured intermediate representation to Mermaid syntax with color styling.

3.5 JSON Storage and Metadata

Each process diagram is stored as a JSON object that includes:

1. **Core fields (required):**
   - `id`: unique identifier
   - `title`: human-readable name
   - `description`: short textual description
   - `mermaid`: Mermaid source string (including color styling)
   - `version`: version number or timestamp
   - `prompt_version`: version identifier for the specific prompt template used (enables reproducibility)
   - `llm_version`: version of the LLM used for generation (e.g., "Gemini 2.0 Flash", "GPT-4")
2. **Metadata fields (optional but recommended):**
   - `domain`: e.g., `biology`, `chemistry`, `mathematics`, `physics`, `computer_science`
   - `category`: subdomain or pathway family
   - `source`: reference to the originating paper, textbook, or URL
   - `entities`: key entities (genes, proteins, molecules, algorithms, etc.)
   - `color_distribution`: count of nodes by color category
   - `annotations`: notes or comments
   - `created_by` / `reviewed_by`: human or AI agents involved
3. **Links:**
   - Links to related diagrams (e.g., upstream or downstream processes)

- Links to external systems (CopernicusAI briefings, metadata database entries, etc.)

This JSON format allows the diagrams to be treated as first-class data objects that can be indexed, searched, and cross-referenced. The schema is intentionally minimal and extensible, allowing domain-specific additions while maintaining core interoperability across disciplines.

Example JSON Schema:


```
{
  "id": "beta-galactosidase-regulation",
  "title": "Beta-Galactosidase Regulation System",
  "description": "Regulatory system controlling lactose metabolism in E. coli",
  "mermaid": "flowchart TD\n    A[Lactose Present] --> B[LacI Repressor]\n    ...",
  "version": "1.2",
  "prompt_version": "v2.1",
  "llm_version": "Gemini 2.0 Flash",
  "domain": "biology",
  "category": "gene_regulation",
  "source": "Lac Operon: A Paradigm of Gene Regulation. Nature Reviews Genetics, 20
  "entities": ["LacI", "beta-galactosidase", "lactose", "cAMP", "CAP"],
  "color_distribution": {
    "red": 2,
    "yellow": 5,
    "green": 8,
    "blue": 4,
    "violet": 3
  },
  "annotations": "Validated against textbook description. Reviewed by domain expert
  "created_by": "LLM (Gemini 2.0 Flash)",
  "reviewed_by": "Domain Expert",
  "links": {
    "related_diagrams": ["lactose-transport", "glucose-metabolism"],
    "external": {
      "glmp": "https://huggingface.co/spaces/garywelz/glmp",
      "source_paper": "https://doi.org/10.1038/nrg1234"
    }
  }
}
```


3.6 Iterative Refinement Workflow

The initial LLM-produced diagram is rarely perfect. The Framework therefore treats each diagram as a draft subject to iterative improvement:

1. **AI Consensus Step (Optional but Recommended):**
   - A second LLM (a "critic" model) reviews the first LLM's diagram against the source text
   - The critic model flags potential errors, missing steps, or inconsistencies
   - This reduces the burden on human reviewers by catching obvious errors before expert review
   - Example critic prompt: "Review this process diagram for accuracy. Compare it to the source text and identify any missing steps, incorrect relationships, or logical inconsistencies."

2. **Review loop:**
    - Human reviewer checks correctness against source
    - Reviewer flags errors (missing steps, incorrect branches, color misassignments, oversimplifications)
    - Reviewer either edits the Mermaid directly or uses guided prompts to ask the LLM for revisions
3. **Validation prompts:**
    - "Compare this diagram to the source text and list any missing steps."
    - "Identify any logical inconsistencies between the diagram and the description."
    - "Verify that all color assignments follow the Programming Framework color system."
4. **Versioning:**
    - Each set of changes increments the `version` field
    - Past versions are retained for audit and comparison

Over time, this creates a library of increasingly accurate diagrams with traceable revision histories. The AI consensus step reduces human labor cost while maintaining quality through multi-model validation.

Validation Methodology:

The Framework employs a multi-stage validation process to ensure diagram accuracy:

1. **Initial Validation:**
    - **Who validates:** Domain experts (for high-stakes scientific content) or knowledgeable reviewers (for educational content)
    - **Criteria for correctness:**
        - All steps from source text are represented
        - Branching logic matches source description
        - Color assignments follow Framework conventions (or documented domain-specific customizations)
        - No hallucinated steps beyond source material
        - Logical flow is consistent (no unreachable nodes, proper entry/exit points)
2. **Structured Validation Checklist:**
    - Completeness: Are all key steps from the source represented?
    - Accuracy: Do the relationships and flows match the source description?
    - Consistency: Are color assignments consistent throughout?
    - Clarity: Is the diagram readable and interpretable?
3. **Agreement Metrics (Future Work):**
    - Current implementation relies on single-reviewer validation
    - Future work should include inter-rater reliability studies
    - Multiple reviewers could independently validate diagrams, with agreement metrics (e.g., Cohen's kappa) calculated for correctness assessments
4. **Stopping Conditions:**
    - Validation continues until reviewer confirms diagram accuracy
    - For high-stakes content, multiple review cycles may be required
    - Version history tracks all changes, enabling rollback if errors are discovered later
5. **Provenance Tracking:**
    - Each diagram includes metadata about who created, reviewed, and validated it
    - Source references enable verification against original materials
    - Version history provides audit trail for all modifications

---

4. Applications Across Disciplines

The Programming Framework has been successfully applied across multiple scientific domains. Here we describe applications in biology, chemistry, mathematics, physics, and computer science.

4.1 Biological Processes: Genome Logic Modeling Project (GLMP)

The most extensive application to date is in the Genome Logic Modeling Project (GLMP), which applies the Framework to map biological processes.

Scope: - 100+ processes analyzed across multiple organisms and systems - Processes span 6+ major categories: Central Dogma, Metabolism, Signaling, Proteins, Photosynthesis, DNA Repair - Comprehensive coverage of regulatory networks, metabolic pathways, and cellular processes

Data sources: - Textbook descriptions - Review articles - Online open-education resources - Primary research papers

Pipeline: 1. Select a process (e.g., "beta-galactosidase regulation") 2. Identify canonical description and supporting references 3. Run the Programming Framework pipeline to generate initial Mermaid diagram with color coding 4. Refine the diagram in collaboration with domain knowledge 5. Store final diagram and metadata in JSON in Google Cloud Storage 6. Expose diagrams via web-based interactive viewer

Outcomes: - A coherent collection of biological process diagrams with consistent style - A living demonstration of the Framework's applicability in a complex scientific domain - Integration with the broader CopernicusAI Knowledge Engine

Example: The beta-galactosidase regulation system in E. coli demonstrates complex regulatory logic with environmental inputs (Red), regulatory proteins and enzymes (Yellow), metabolic reactions (Green), intermediate states (Blue), and final products (Violet). The color coding enables rapid identification of regulatory checkpoints and feedback mechanisms.

Sample Diagram - Beta-Galactosidase Regulation:

Figure 2: Beta-Galactosidase Regulation System in E. coli. Note: This is one possible representation of the beta-galactosidase regulation process; more detailed versions showing additional regulatory steps, feedback mechanisms, or molecular interactions can be generated by specifying greater detail in the prompt. Interactive versions are available at https://huggingface.co/spaces/garywelz/glmp.

Color Legend: Red (Triggers/Inputs), Yellow (Structures/Objects), Green (Processing/Operations), Blue (Intermediates/States), Violet (Products/Outputs). Interactive versions of this and other biological process diagrams are available at the GLMP Hugging Face Space.

Mermaid Markdown Code:

```
flowchart TD
    A[Lactose Present] --> B[LacI Repressor]
    B --> C{Lactose Binding}
    C -->|Yes| D[Repressor Released]
    C -->|No| E[Repressor Bound]
    D --> F[RNA Polymerase]
    E --> G[No Transcription]
    F --> H[Transcription Initiated]
    H --> I[mRNA Production]
```

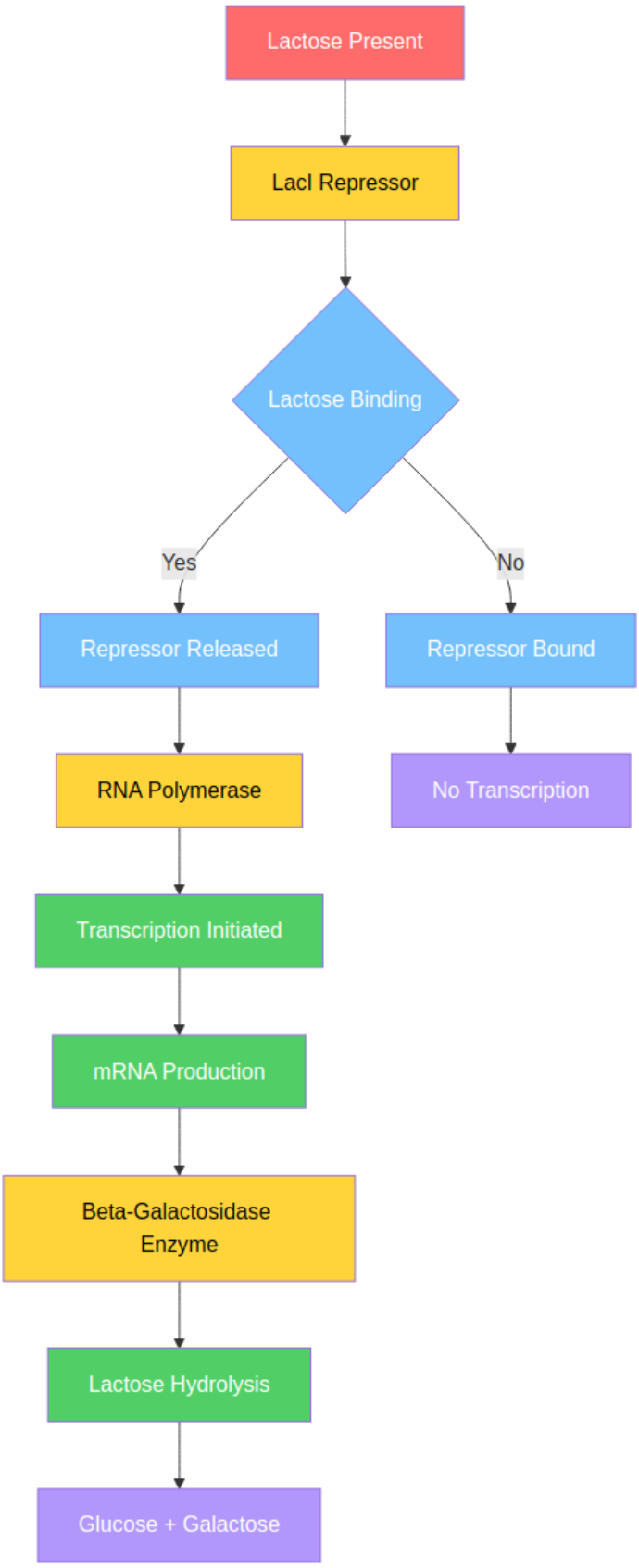

Figure 2: Figure 2: Beta-Galactosidase Regulation System

```
    I --> J[Beta-Galactosidase Enzyme]
    J --> K[Lactose Hydrolysis]
    K --> L[Glucose + Galactose]

    style A fill:#ff6b6b,color:#fff
    style B fill:#ffd43b,color:#000
    style C fill:#74c0fc,color:#fff
    style D fill:#74c0fc,color:#fff
    style E fill:#74c0fc,color:#fff
    style F fill:#ffd43b,color:#000
    style G fill:#b197fc,color:#fff
    style H fill:#51cf66,color:#fff
    style I fill:#51cf66,color:#fff
    style J fill:#ffd43b,color:#000
    style K fill:#51cf66,color:#fff
    style L fill:#b197fc,color:#fff
```

4.2 Chemical Processes

The Framework has been applied to chemical reactions and processes across major branches of chemistry, including organic reactions, synthesis pathways, thermodynamic processes, catalysis mechanisms, and electrochemical processes. The methodology demonstrates transferability from biological to chemical domains, with 70+ processes mapped across 14 batches.

Sample Diagram - Catalytic Hydrogenation:

Figure 3: Catalytic Hydrogenation Reaction Process. Note: This is one possible representation; more detailed versions showing molecular structures, energy diagrams, reaction mechanisms, or kinetic steps can be generated by specifying greater detail in the prompt.

Color Legend: Red (Reactants/Inputs), Yellow (Catalyst/Structure), Green (Reactions/Operations), Blue (Intermediates), Violet (Products)

Mermaid Markdown Code:

```
flowchart TD
    A[Alkene Substrate] --> B[Catalyst: Pd/C]
    C[H2 Gas] --> B
    B --> D{Catalyst Activation}
    D -->|Active| E[H2 Adsorption]
    D -->|Inactive| F[No Reaction]
    E --> G[Substrate Binding]
    G --> H[Hydrogen Addition]
    H --> I[Intermediate Complex]
    I --> J[Product Release]
    J --> K[Alkane Product]
    K --> L[Catalyst Regeneration]

    style A fill:#ff6b6b,color:#fff
    style C fill:#ff6b6b,color:#fff
```

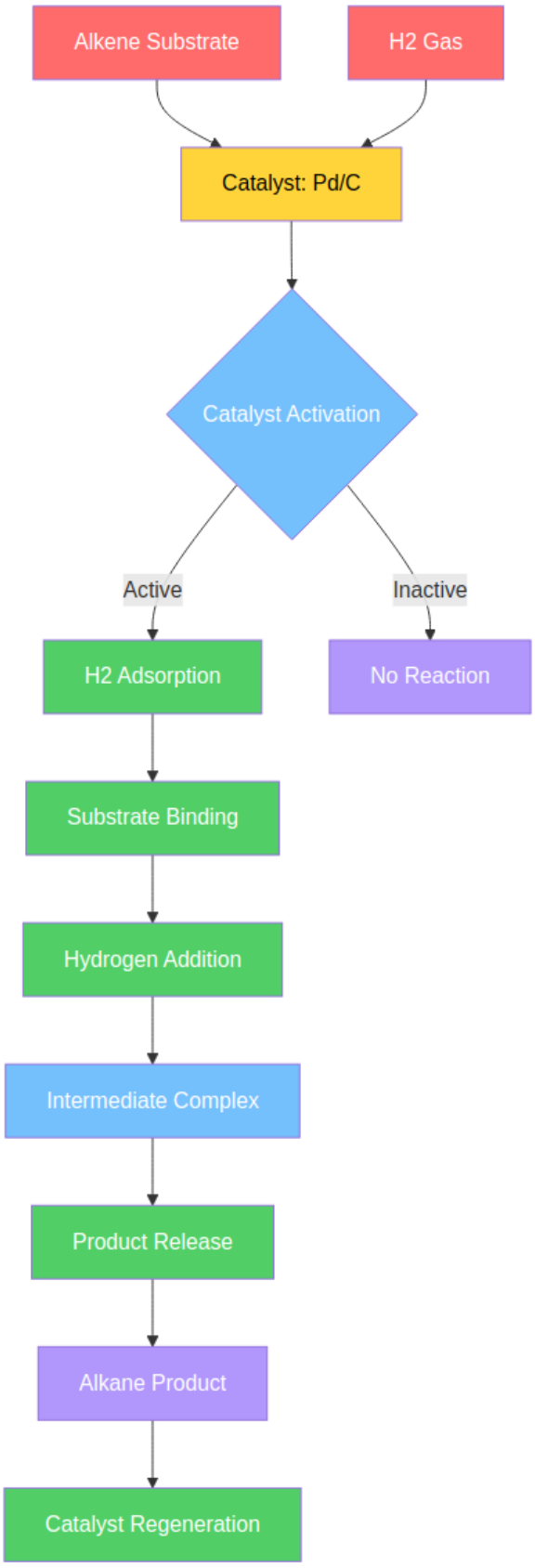

Figure 3: Figure 3: Catalytic Hydrogenation Reaction

```
    style B fill:#ffd43b,color:#000
    style D fill:#74c0fc,color:#fff
    style E fill:#51cf66,color:#fff
    style F fill:#b197fc,color:#fff
    style G fill:#51cf66,color:#fff
    style H fill:#51cf66,color:#fff
    style I fill:#74c0fc,color:#fff
    style J fill:#51cf66,color:#fff
    style K fill:#b197fc,color:#fff
    style L fill:#51cf66,color:#fff
```

4.3 Mathematical Processes

The Framework has been applied to mathematical processes including proof methods, algorithms, and computational workflows. The Euclidean algorithm example demonstrates visualization of mathematical processes with inputs (Red), mathematical structures (Yellow), computational operations (Green), intermediate states (Blue), and final products (Violet).

Sample Diagram - Euclidean Algorithm:

Figure 4: Euclidean Algorithm Process Flow. Note: This is one possible representation; more detailed versions showing proof steps, complexity analysis, or extended Euclidean algorithm variants can be generated by specifying greater detail in the prompt.

Color Legend: Red (Inputs), Blue (Decision/Intermediate States), Green (Operations), Violet (Output)

Mermaid Markdown Code:

```
flowchart TD
    A[Input: a, b] --> B{Is b = 0?}
    B -->|Yes| C[GCD = a]
    B -->|No| D[Calculate r = a mod b]
    D --> E[Set a = b]
    E --> F[Set b = r]
    F --> B
    C --> G[Return GCD]

    style A fill:#ff6b6b,color:#fff
    style B fill:#74c0fc,color:#fff
    style C fill:#74c0fc,color:#fff
    style D fill:#51cf66,color:#fff
    style E fill:#51cf66,color:#fff
    style F fill:#51cf66,color:#fff
    style G fill:#b197fc,color:#fff
```

4.4 Physics Processes

The Framework has been applied to physical processes including quantum mechanics, nuclear physics, electromagnetism, thermodynamics, and particle physics. Physical phenomena can be represented with energy inputs (Red), wave functions and fields (Yellow), quantum processing (Green), intermediate states (Blue), and final products (Violet).

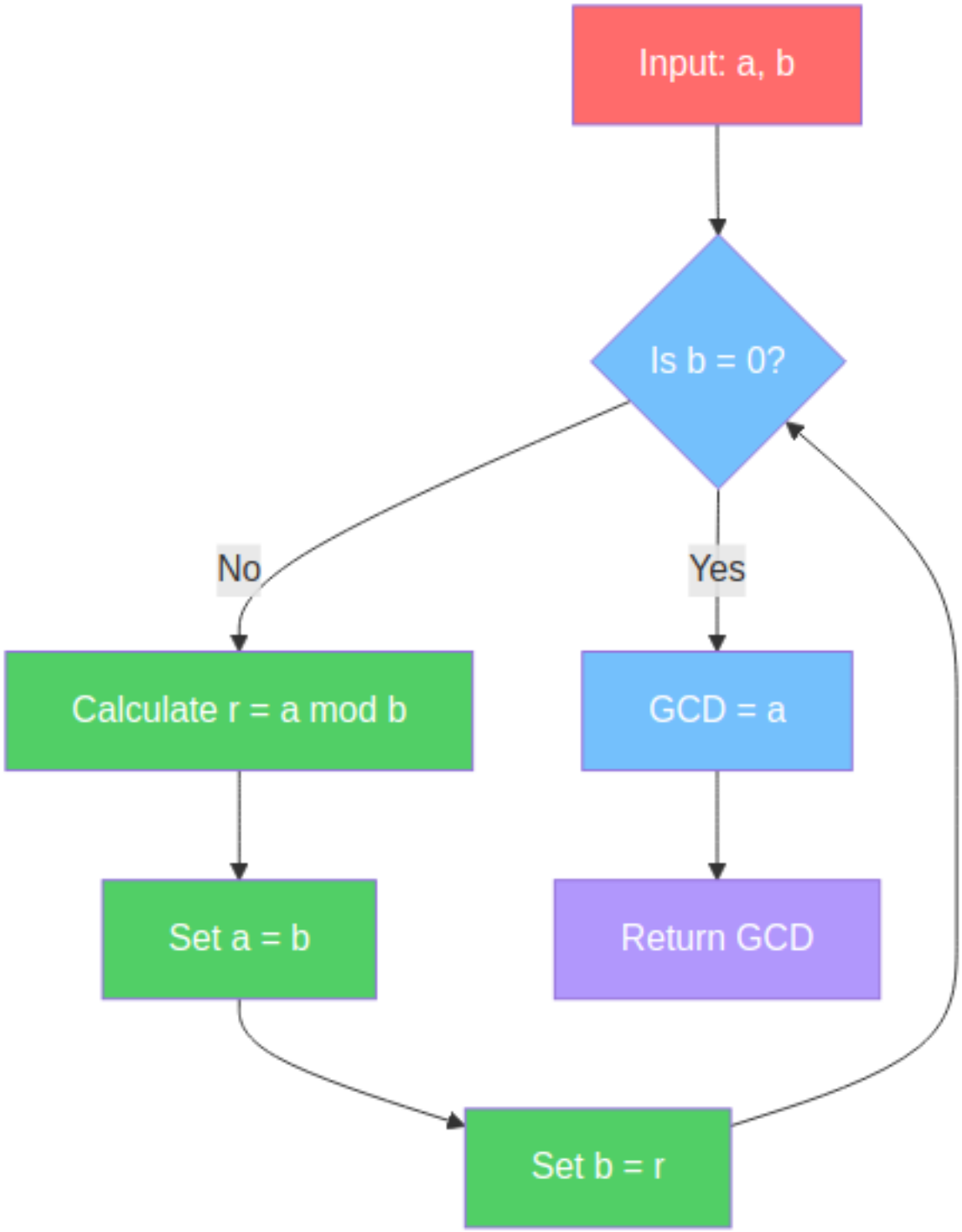

Figure 4: Figure 4: Euclidean Algorithm

4.5 Computer Science Processes

The Framework has been applied to computational processes including:

Algorithms & Data Structures: Sorting algorithms, graph algorithms, search methods
Software Engineering: Design patterns, architecture patterns, development workflows
Artificial Intelligence: Machine learning pipelines, neural network training, inference processes
Computer Security: Cryptographic processes, authentication workflows, security protocols
Computer Networks: Protocol implementations, distributed systems, network routing
Database Systems: Query processing, transaction management, data structures
Computer Graphics: Rendering pipelines, game development workflows, graphics algorithms

Example: Computational workflows demonstrate how software processes can be visualized with data inputs (Red: input data, parameters), data structures and algorithms (Yellow: data structures, algorithm designs), computational operations (Green: processing steps, transformations), intermediate states (Blue: temporary data, state variables), and final outputs (Violet: results, reports, visualizations).

4.6 Comparison with Existing Visualization Tools

The Programming Framework addresses gaps in existing process visualization tools across scientific domains. This section provides a consolidated comparison highlighting common themes and domain-specific considerations.

Domain-Specific Standards and Tools:

Domain	Standards	Key Tools	Primary Focus
Biology	SBGN [5], BioPAX [6], SBML [7]	CellDesigner [8], Cytoscape [9], PathVisio, VANTED	Precise biochemical notation, quantitative modeling
Chemistry	None (lacks unified standard)	ChemDraw, ChemDoodle, ChemSketch	Molecular structure drawing
Mathematics	None (lacks process visualization standard)	Coq, Isabelle, Lean (proof systems); MATLAB, Mathematica (computation)	Formal proof, computation
Physics	None (lacks unified standard)	MATLAB, COMSOL, ANSYS, OpenFOAM	Numerical simulation
Computer Science	UML [10]	Enterprise Architect, Visual Paradigm, Lucidchart, draw.io, PlantUML	Software design, architecture
The Programming Framework	Mermaid Markdown (text-based standard)	LLM + Mermaid.js	Cross-disciplinary rapid visualization, accessible process representation

Common Limitations of Existing Tools:

1. **Cost Barriers:** Many professional tools require expensive licenses (CellDesigner, Enterprise Architect, Visual Paradigm, commercial Cytoscape plugins)

2. **Complexity:** Steep learning curves and specialized training requirements
3. **Software Dependencies:** Require installation of specific software packages or cloud subscriptions
4. **Format Lock-in:** Proprietary or specialized formats that limit portability
5. **Domain Isolation:** Tools are typically domain-specific, preventing cross-domain comparison
6. **Limited Accessibility:** Binary formats or complex interfaces create barriers to entry

Advantages of the Programming Framework's Mermaid-Based Approach:

1. **Accessibility:**
   - Text-based format readable by humans without specialized software
   - Can be edited in any text editor
   - No software installation required for viewing (renders in GitHub, documentation systems, web browsers)
2. **Version Control:**
   - Native compatibility with Git and other version control systems
   - Enables collaborative editing and change tracking
   - Supports branching and merging workflows
3. **Low Barrier to Entry:**
   - Minimal learning curve compared to specialized tools
   - No licensing costs
   - Works on any platform with a web browser
4. **Integration:**
   - Easy integration into documentation, websites, and web applications
   - Can be embedded in Markdown documents, Jupyter notebooks, and documentation systems
   - JSON storage enables programmatic access and integration
5. **Cross-Domain Consistency:**
   - Same format works across biology, chemistry, mathematics, physics, and computer science
   - Enables comparison and integration across disciplines
   - Suggested color semantics can facilitate pattern recognition when consistently applied
6. **Rapid Prototyping:**
   - Fast iteration cycles for exploring process representations
   - LLM-assisted generation reduces initial authoring time
   - Easy to modify and refine

Complementary Use Cases:

The Programming Framework is not intended to replace specialized tools for high-precision quantitative modeling, formal verification, or standards compliance. Instead, it serves complementary purposes:

- **Educational contexts:** Quick visualization for teaching and learning
- **Rapid exploration:** Initial process mapping before detailed modeling
- **Cross-disciplinary comparison:** Comparing processes across domains using consistent representation
- **Documentation:** Embedding process diagrams in documentation and web content
- **Collaborative workflows:** Version-controlled, text-based collaboration
- **Integration:** Embedding process visualizations in knowledge engines and research platforms

For researchers requiring precise biochemical notation (SBGN), quantitative modeling, formal proof verification, or full UML notation, specialized tools remain the appropriate choice. The Programming Framework offers an accessible alternative for contexts where rapid visualization, cross-domain comparison, and ease of use are priorities.

5. Results and Discussion

This section summarizes qualitative and quantitative observations from using the Programming Framework across multiple domains.

5.1 Process Coverage

The Framework has been used to produce:

1. **Biology:** 100+ diagrams via GLMP spanning multiple pathway families and organisms
2. **Chemistry:** 70+ processes across 14 batches covering all major branches of chemistry
3. **Mathematics:** 5+ processes demonstrating proof methods, algorithms, and computational workflows
4. **Physics:** 5+ processes covering quantum mechanics, nuclear physics, electromagnetism, and thermodynamics
5. **Computer Science:** 21+ processes across 7 batches covering algorithms, software engineering, AI, security, networks, databases, and graphics

This demonstrates that the method is flexible enough to handle diverse content across scientific disciplines, given appropriate prompts and scoping.

5.2 Benefits Observed

1. **Standardized representation:**
   - Using Mermaid provides a simple, readable, and widely supported format
   - The five-category color system enables rapid visual parsing and cross-domain comparison
   - Textual diagrams facilitate version control and collaboration
2. **Reduced authoring burden:**
   - LLMs handle the first-pass extraction and drafting
   - Humans focus on validation and refinement, not manual diagram creation from scratch
   - Color coding reduces cognitive load in understanding complex processes
3. **Improved comparability:**
   - Processes in different domains share a common visual language
   - Color semantics enable pattern recognition across disciplines
   - JSON metadata enables filtering and grouping by domain, category, or entities
4. **Cross-disciplinary insights:**
   - When a consistent color scheme is applied, it can reveal structural similarities between processes in different fields
   - Researchers can identify optimization opportunities by comparing processes across domains
   - The framework enables systematic analysis of process efficiency and bottlenecks
5. **Reusability and integration:**
   - Diagrams can be linked to podcasts, papers, and metadata records
   - The same representation can support multiple interfaces (web viewer, documentation, educational materials)
   - JSON format enables programmatic access and integration with other tools

5.3 Limitations and Failure Modes

Despite its benefits, the Framework has clear limitations:

1. **LLM hallucinations and shortcuts:**
    - The model may infer steps not present in the source text
    - It may oversimplify or omit important branches when the source is ambiguous
    - Color assignments may be inconsistent without careful prompt engineering
2. **Granularity mismatch:**
    - Without careful scoping, diagrams may be too coarse (missing key detail) or too fine-grained (overwhelming users)
    - Different domains may require different levels of detail
3. **Domain-specific nuance:**
    - Some fields require notations or conventions that Mermaid cannot fully express
    - For biology, Mermaid lacks the precision of SBGN [5] or the quantitative modeling capabilities of specialized tools like CellDesigner [8] and Cytoscape [9]
    - Subtle mechanistic distinctions can be lost in generic flowchart form
    - Color semantics may need domain-specific interpretation
    - The Framework is not a replacement for specialized tools when precision, quantitative modeling, or standards compliance are required
4. **Human time cost for validation:**
    - While authoring is faster, thorough validation still requires expert review
    - For high-stakes scientific use, domain experts must be involved
    - Color assignment verification requires domain knowledge
5. **Quantitative evaluation limitations:**
    - The current work provides qualitative demonstrations across multiple domains but lacks systematic quantitative validation
    - No formal accuracy metrics comparing LLM-generated diagrams to expert-created ground truth
    - No inter-rater reliability studies for diagram correctness
    - Future work should include validation studies with domain experts and quantitative accuracy measurements

Mitigation Strategies: The Framework addresses these limitations through systematic mitigation strategies: (1) Multi-checker workflows using multiple AI models for cross-validation, (2) Human-in-the-loop review processes with domain experts, (3) Iterative refinement cycles that improve accuracy over time, (4) Version control and provenance tracking to maintain audit trails, and (5) Structured validation prompts that systematically check for common error patterns. These strategies reduce but do not eliminate the need for careful validation, particularly for high-stakes scientific applications.

5.4 Ethical Considerations

The use of LLMs for scientific process extraction raises several ethical considerations that must be addressed:

1. **Accuracy and Responsibility:**
    - LLM-generated diagrams may contain errors or hallucinations that could mislead researchers or students
    - The Framework mitigates this through human validation, but users must understand that diagrams require expert review before use in high-stakes contexts
    - Responsibility for scientific accuracy ultimately lies with human validators, not the AI system
2. **Bias in Process Extraction:**
    - LLMs may reflect biases present in training data, potentially emphasizing certain process aspects while de-emphasizing others
    - Domain-specific knowledge may be underrepresented in training data, leading to incomplete or

culturally biased representations
    - Validation by domain experts from diverse backgrounds helps mitigate these concerns
3. **Accessibility and Equity:**
    - The Framework's text-based approach improves accessibility compared to proprietary tools
    - However, color-coding may present challenges for colorblind users (addressed through customization options)
    - Future work should include systematic evaluation of accessibility barriers
4. **Intellectual Property:**
    - Diagrams derived from copyrighted source materials must respect original authorship
    - The Framework's metadata system tracks sources, enabling proper attribution
    - Users should ensure compliance with copyright and fair use guidelines
5. **Reproducibility:**
    - LLM outputs can vary across model versions and prompt variations
    - The Framework addresses this through version control and prompt documentation (see Appendix A)
    - Users should document LLM versions and prompt configurations for reproducibility
6. **Scientific Misinformation:**
    - Incorrect process diagrams could propagate scientific misinformation
    - The Framework's validation requirements and version control help prevent this
    - Users should clearly indicate validation status and source materials

Recommendations: - Always validate LLM-generated diagrams with domain experts before publication or high-stakes use - Maintain clear provenance and source attribution - Document validation status and reviewer qualifications - Consider accessibility needs when using color-coding - Use version control to enable error correction and rollback

5.5 Design Choices and Trade-Offs

1. **Mermaid vs. other formats:**
    - **Chosen for:** Simplicity, human-readability, broad tooling support, version control compatibility
    - **Trade-off:** Lacks the full expressivity of specialized diagramming languages (e.g., SBGN for biology)
2. **LLM-driven vs. rule-based extraction:**
    - **Chosen for:** Flexibility across domains without bespoke parsers
    - **Trade-off:** Behavior is probabilistic and must be constrained by prompts and validation
3. **Suggested color system:**
    - **Chosen for:** Provides a starting point for consistent visualization, simplicity, memorability
    - **Trade-off:** May not suit all processes; domains may require custom schemes (as demonstrated in GLMP with logical connectives)
    - **Flexibility:** The Framework allows customization, recognizing that different processes benefit from different color assignments
4. **JSON storage:**
    - **Chosen for:** Interoperability and ease of integration
    - **Trade-off:** Does not inherently enforce schema constraints without additional tooling

---

6. Future Directions

Several directions extend beyond the current implementation, organized by timeline:

Short-term (6-12 months): 1. **Stronger validation tooling:** - Automated checks for structural consistency (e.g., no unreachable nodes, single entry/exit) - Rule-based validators for specific domains (e.g., biochemical constraints, conservation laws) - Color assignment validation against domain-specific rules

2. **Interactive editors:**
    - Web interfaces where users can edit Mermaid diagrams directly and see live previews
    - Assisted editing with AI suggestions for color assignments and process improvements
    - Collaborative editing with version control

Medium-term (1-2 years): 3. **Formal semantics:** - Mapping Mermaid diagrams to more formal representations (e.g., Petri nets, state machines) for analysis - Enabling automated reasoning about process properties - Supporting process simulation and optimization

4. **Integration with knowledge graphs:**
    - Linking diagram nodes to entities in a knowledge graph (genes, chemicals, algorithms, etc.)
    - Supporting multi-diagram reasoning and comparison
    - Enabling cross-domain pattern discovery

Long-term (2+ years): 5. **Community-driven libraries:** - Public repositories where researchers can submit, review, and reuse process diagrams - Versioned process libraries for specific domains - Community standards for color assignment in specialized domains

6. **Domain-specific extensions:**
    - Specialized color schemes for specific domains (e.g., extended biology color scheme)
    - Domain-specific validation rules
    - Integration with domain-specific tools and databases

---

7. Conclusion

The Programming Framework offers a practical, reusable method for turning textual process descriptions into structured, computable diagrams using LLMs and Mermaid, enhanced by a suggested five-category color-coding system that can be customized for specific domains. It has been successfully applied to biological pathways (via GLMP), chemical reactions, mathematical algorithms, physical processes, and computational workflows, demonstrating both domain-agnostic flexibility and the need for careful human validation.

The framework's suggested color-coding system provides a starting point for consistent visualization, but recognizes that different processes may benefit from custom color schemes, as demonstrated by GLMP's use of domain-specific colors for logical connectives in biological regulatory networks.

Rather than claiming to "solve" process modeling, the Framework is proposed as infrastructure: a starting point that others can adopt, critique, and extend. By providing a clear pipeline—from text to Mermaid to JSON, with iterative refinement and a suggested color-coding system that can be customized—the Programming Framework lowers the barrier to creating and maintaining high-quality process representations across scientific disciplines.

We invite researchers, educators, and practitioners to experiment with the method, contribute process examples, and help refine both the prompts and the underlying schemas. In the broader CopernicusAI Knowledge

Engine, the Programming Framework plays a central role in making processes visible, comparable, and computable—an essential step toward more transparent and navigable scientific workflows.

Availability: The Programming Framework methodology, examples, and tools are available at: https://huggingface.co/spaces/garywelz/programming_framework

Related Work: The Genome Logic Modeling Project (GLMP) demonstrates the Framework's application to biology: https://huggingface.co/spaces/garywelz/glmp

---

References

[1] Sveidqvist, K. (2014). Mermaid: Diagramming and charting software. GitHub Repository. https://github.com/mermaid-js/mermaid

[2] Wikipedia contributors. (2025). Mermaid (software). In *Wikipedia, The Free Encyclopedia*. Retrieved from https://en.wikipedia.org/wiki/Mermaid_(software)

[3] Lardinois, F. (2024, March 20). Mermaid Chart, a Markdown-like tool for creating diagrams, raises $7.5M. *TechCrunch*. https://techcrunch.com/2024/03/20/mermaid-chart-a-markdown-like-tool-for-creating-diagrams-raises-7-5m/

[4] Pot, J. (2024, November 13). Use Mermaid to Create Charts and Diagrams Without Image Editing Tools. *LifeHacker*. https://lifehacker.com/use-mermaid-to-create-charts-and-diagrams-without-image-1851201234

[5] Le Novère, N., Hucka, M., Mi, H., Moodie, S., Schreiber, F., Sorokin, A., … & Kitano, H. (2009). The Systems Biology Graphical Notation. *Nature Biotechnology*, 27(8), 735-741. https://doi.org/10.1038/nbt.1558

[6] Demir, E., Cary, M. P., Paley, S., Fukuda, K., Lemer, C., Vastrik, I., … & Bader, G. D. (2010). The BioPAX community standard for pathway data sharing. *Nature Biotechnology*, 28(9), 935-942. https://doi.org/10.1038/nbt.1666

[7] Hucka, M., Finney, A., Sauro, H. M., Bolouri, H., Doyle, J. C., Kitano, H., … & Wang, J. (2003). The systems biology markup language (SBML): a medium for representation and exchange of biochemical network models. *Bioinformatics*, 19(4), 524-531. https://doi.org/10.1093/bioinformatics/btg015

[8] Funahashi, A., Matsuoka, Y., Jouraku, A., Morohashi, M., Kikuchi, N., & Kitano, H. (2008). CellDesigner 3.5: A versatile modeling tool for biochemical networks. *Proceedings of the IEEE*, 96(8), 1254-1265. https://doi.org/10.1109/JPROC.2008.925458

[9] Shannon, P., Markiel, A., Ozier, O., Baliga, N. S., Wang, J. T., Ramage, D., … & Ideker, T. (2003). Cytoscape: a software environment for integrated models of biomolecular interaction networks. *Genome Research*, 13(11), 2498-2504. https://doi.org/10.1101/gr.1239303

[10] Object Management Group. (2017). *Unified Modeling Language (UML), Version 2.5.1*. https://www.omg.org/spec/UML/2.5.1/

[11] Berners-Lee, T., Hendler, J., & Lassila, O. (2001). The semantic web. *Scientific American*, 284(5), 34-43. https://doi.org/10.1038/scientificamerican0501-34

[12] Gruber, T. R. (1993). A translation approach to portable ontology specifications. *Knowledge Acquisition*, 5(2), 199-220. https://doi.org/10.1006/knac.1993.1008

[13] Noy, N. F., & McGuinness, D. L. (2001). Ontology development 101: A guide to creating your first ontology. *Stanford Knowledge Systems Laboratory Technical Report KSL-01-05*. https://protege.stanford.edu/publications/ontology_development/ontology101.pdf

[14] Uschold, M., & Gruninger, M. (1996). Ontologies: Principles, methods and applications. *Knowledge Engineering Review*, 11(2), 93-136. https://doi.org/10.1017/S0269888900007797

[15] Larkin, J. H., & Simon, H. A. (1987). Why a diagram is (sometimes) worth ten thousand words. *Cognitive Science*, 11(1), 65-100. https://doi.org/10.1016/S0364-0213(87)80026-5

[16] Healey, C. G., & Enns, J. T. (2012). Attention and visual memory in visualization and computer graphics. *IEEE Transactions on Visualization and Computer Graphics*, 18(7), 1170-1188. https://doi.org/10.1109/TVCG.2011.127

[17] Brown, T., Mann, B., Ryder, N., Subbiah, M., Kaplan, J. D., Dhariwal, P., ... & Amodei, D. (2020). Language models are few-shot learners. *Advances in Neural Information Processing Systems*, 33, 1877-1901. https://proceedings.neurips.cc/paper/2020/hash/1457c0d6bfcb4967418bfb8ac142f64a-Abstract.html

[18] OpenAI. (2023). GPT-4 Technical Report. *arXiv preprint arXiv:2303.08774*. https://arxiv.org/abs/2303.08774

[19] Zelenko, D., Aone, C., & Richardella, A. (2003). Kernel methods for relation extraction. *Journal of Machine Learning Research*, 3, 1083-1106. https://www.jmlr.org/papers/v3/zelenko03a.html

[20] Ahn, D. (2006). The stages of event extraction. *Proceedings of the Workshop on Annotating and Reasoning about Time and Events*, 1-8. https://aclanthology.org/W06-0901/

[21] Ji, S., Pan, S., Cambria, E., Marttinen, P., & Yu, P. S. (2021). A survey on knowledge graphs: Representation, acquisition, and applications. *IEEE Transactions on Neural Networks and Learning Systems*, 33(2), 494-514. https://doi.org/10.1109/TNNLS.2021.3070843

[22] Schulman, J., Zoph, B., Kim, C., Hilton, J., Menick, J., Weng, J., ... & Fedus, W. (2022). ChatGPT: Optimizing language models for dialogue. OpenAI Blog. https://openai.com/blog/chatgpt/

[23] White, J., Fu, Q., Hays, S., Sandborn, M., Olea, C., Gilbert, H., ... & Schmidt, D. C. (2023). A prompt pattern catalog to enhance prompt engineering with ChatGPT. *arXiv preprint arXiv:2302.11382*. https://arxiv.org/abs/2302.11382

[24] Kong, N., Agrawala, M., & Hartmann, B. (2019). Perfopticon: Visual querying for performance in large-scale graph databases. *Proceedings of the 2019 CHI Conference on Human Factors in Computing Systems*, 1-12. https://doi.org/10.1145/3290605.3300688

[25] Liu, C., Wu, P., Li, C., & Zhu, J. (2020). Generating diagrams from natural language. *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing*, 3637-3647. https://doi.org/10.18653/v1/2020.emnlp-main.295

Prior Work and Related Artifacts

The following citations reference publicly available implementations and demonstrations of the Programming Framework methodology:

Welz, G. (2024–2025). *The Programming Framework: A Universal Method for Process Analysis*. Hugging Face Spaces. https://huggingface.co/spaces/garywelz/programming_framework

Welz, G. (2024–2025). *GLMP: Genome Logic Modeling Project - A Microscope for Biological Processes*. Hugging Face Spaces. https://huggingface.co/spaces/garywelz/glmp

Welz, G. (2024). *From Inspiration to AI: Biology as Visual Programming*. Medium. https://medium.com/@garywelz_47126/fro
inspiration-to-ai-biology-as-visual-programming-520ee523029a

Note on Diagram Rendering: All Mermaid diagram code blocks in this paper render as interactive flowchart diagrams in HTML/web versions. The Programming Framework Hugging Face Space (https://huggingface.co/spaces/garywelz/programming_framework) and GLMP Space (https://huggingface.co/spaces/garywelz/glmp) provide live, interactive examples where readers can view rendered diagrams. For PDF versions, the Mermaid code is included for reference and can be rendered using the Mermaid Live Editor (https://mermaid.live) or any Mermaid-compatible viewer.

Appendix A: Complete Prompt Template

The following is a complete example of a prompt used for process extraction. The prompt is formatted to fit within standard page margins:

Task Statement: You are a scientific process modeler. Given a textual description of a process, extract a sequence of steps, decisions, and flows. Classify each element according to the Programming Framework's suggested color system.

Color System: - Red (#ff6b6b): Triggers & Inputs - Initial conditions, environmental inputs, starting materials, or data inputs - Yellow (#ffd43b): Structures & Objects - Physical structures, molecules, data structures, algorithms, or logical constructs - Green (#51cf66): Processing & Operations - Transformations, reactions, computations, or operations that change state - Blue (#74c0fc): Intermediates & States - Intermediate products, temporary states, or transitional conditions - Violet (#b197fc): Products & Outputs - Final outputs, end products, or results

Output Format (JSON):

```
{
  "steps": [
    {
      "id": "step_1",
      "type": "trigger|structure|operation|intermediate|product",
      "label": "Concise description",
      "color": "red|yellow|green|blue|violet",
      "inputs": ["entity1", "entity2"],
      "outputs": ["entity3"]
    }
  ],
  "edges": [
    {
      "from": "step_1",
      "to": "step_2",
      "label": "optional edge label"
    }
  ]
}
```

Constraints: - No speculative steps beyond the text - Use consistent terminology for entities - Explicitly

represent branching conditions - Maintain color consistency throughout

Example:

Input: “When lactose is present, the Lac repressor releases from the operator, allowing RNA polymerase to bind and transcribe the lac operon genes.”

Output:

```
{
  "steps": [
    {"id": "s1", "type": "trigger", "label": "Lactose Present", "color": "red"},
    {"id": "s2", "type": "operation", "label": "Repressor Release", "color": "green
    {"id": "s3", "type": "structure", "label": "RNA Polymerase", "color": "yellow"}
    {"id": "s4", "type": "operation", "label": "Transcription", "color": "green"},
    {"id": "s5", "type": "product", "label": "lac Operon mRNA", "color": "violet"}
  ],
  "edges": [
    {"from": "s1", "to": "s2"},
    {"from": "s2", "to": "s3"},
    {"from": "s3", "to": "s4"},
    {"from": "s4", "to": "s5"}
  ]
}
```

Final Instruction: Now analyze the following process description: [PROCESS_TEXT_HERE]

Note on Prompt Sensitivity: Prompt design significantly influences extraction fidelity. Small variations in wording, example selection, or constraint specification can yield substantially different outputs. The Framework addresses this through iterative refinement and validation workflows, acknowledging that prompt engineering is an ongoing process rather than a one-time configuration.

Author Note: This work is part of the CopernicusAI Knowledge Engine project, which aims to create AI-powered tools for scientific research synthesis and knowledge discovery. The Programming Framework serves as a foundational component enabling structured representation of processes across scientific disciplines.